Lecture Notes: OpenMP Performance — A Case Study with Monte Carlo π

** Lecture Notes --- HPC 1 — Università di Trieste, A.Y. 2025/2026 **
** Luca Tornatore, luca.tornatore@inaf.it**

Disclaimer: all the codes, the accompanying plots, the technical explanations and the concept are from the Author. The text has been re-assembled from sparse materials and transcripts of lectures, with a detailed prompt by Claude, and revised by the Author.

Lecture Notes: OpenMP Performance — A Case Study with Monte Carlo π
- What this lecture is about
- 1. The algorithm: Monte Carlo estimation of π
- 2. Step zero: the random number generator problem
  - Why `drand48()` is broken with threads
  - The fix: `erand48()` with per-thread state
  - A note on the seeding pattern across the suite
- 3. The four implementations
  - 3.1. Version 0: `critical` on every iteration
  - 3.2. Version 0b: `critical` only on hits
  - 3.3. Version 1: `atomic` on every iteration
  - 3.4. Version 2: local accumulators — how it should be done
  - 3.5. Bonus: the `reduction` clause
  - Summary of predictions
- 4. Try it yourselves
- 5. Results from two servers
  - 5.1. What the numbers say
  - 5.2. The two platforms
    - Platform A: 2-socket Broadwell-EP (E5-2697 v4)
    - Platform B: 4-socket Skylake-SP (Gold 5118)
  - 5.3. Why topology matters
  - 5.4. Details to notice in the individual plots
- 6. Hardware mechanisms
  - 6.1. Cache-line bouncing (MESI)
  - Note: False sharing
- 7. Exercises and discussion
  - Exercises
- 8. Summary
  - Reference plots

What this lecture is about

We have a set of six C codes. They all do the same thing — estimate π with a Monte Carlo method — but they differ in how they handle the parallel accumulation of results. The algorithm is trivial on purpose. However, it is embarassingly parallel and a good candidate to illustrate how synchronization affects performance.
The reasons are rooted in the hardware.

Here are the codes, in the order we'll look at them:

File	Strategy	Expected relative performance
`pi_openmp.uncorrect_random.c`	Thread-unsafe vs. thread-safe PRNG	(correctness, not speed)
`pi_openmp.v0.c`	`critical` on every iteration	Worst
`pi_openmp.v0b.c`	`critical` only on hits (≈ π/4 of iterations)	Second worst
`pi_openmp.v1.c`	`atomic` on every iteration	Better
`pi_openmp.v2.c`	Local accumulator + final `atomic`	Best
`pi_openmp.use_reduction.c`	`omp for` with `reduction(+:Nvalid)`	Same as v2

Every code prints a per-thread wall-clock time, so you can directly see whether the threads are balanced or not.

The plan is:

1. Understand the algorithm.
2. Walk through the four implementations, one by one, and predict what should happen.
3. Run them on your laptops. I'll show results from mine.
4. Look at results from two very different servers — and *then* discover that the topology of the machine amplifies the differences in a way you wouldn't necessarily have predicted.
5. You'll run the test on a Leonardo@CINECA node.

---

1. The algorithm: Monte Carlo estimation of π

Consider the unit square [0,1]×[0,1] and the quarter-circle of radius 1 inscribed in it.
The area of the quarter-circle is π/4. If you throw N random points uniformly in the square, the fraction that lands inside the quarter-circle converges to π/4 as N → ∞.

So: generate pairs (x, y) with x, y ∈ [0,1), count how many satisfy $x^2 + y^2 < 1$, call that Nvalid. Then

$$\pi \approx 4 \cdot \frac{N_{\text{valid}}}{N_{\text{total}}}$$

where N_{total} is the total number of points thrown across all threads.

The statistical error goes as 1/√N, which is outrageous for a numerical method — you would never actually compute π this way. But the algorithm is embarrassingly parallel: each random throw is independent, which makes it a clean vehicle for studying synchronisation overhead.

In all the codes, every thread generates N points (not N/nthreads), and the denominator is correspondingly N × nthreads. This means the total work scales linearly with the thread count. If the parallelisation were perfect (zero overhead), the wallclock time
would stay constant. Any deviation from constant wallclock is synchronisation overhead, and that's what we're

after.

This is a ***weak scaling*** test: the problem size grows with the number of threads, and the metric of success is constant execution time.

---

2. Step zero: the random number generator problem

Before we talk about synchronisation, there is a subtler problem that can destroy the *correctness* of the result: thread safety of the PRNG.

`pi_openmp.uncorrect_random.c` illustrates this. It compiles in two modes, controlled by a preprocessor flag.

The unsafe version (default, without `-DUSE_CORRECT_RND`):

```
srand48(SEED*(myid+1));
#define RND_VAL drand48()
```

The safe version (with `-DUSE_CORRECT_RND`):

```
int unsigned short myseeds[3] = {SEED+(myid), SEED^(myid*3+1), SEED&(myid*4+2)};
seed48( myseeds );
#define RND_VAL erand48(myseeds)
```

Why `drand48()` is broken with threads

`drand48()` keeps its state in a single global 48-bit buffer — a static variable inside the C library. When multiple threads call it concurrently, they read-modify-write that buffer without any locking. Two consequences:

1. **Data race**: the interleaving corrupts the linear congruential sequence. It's no longer the deterministic sequence the generator is supposed to produce.
2. **Statistical bias**: the corrupted sequence can have correlations, reduced period, or other defects. With enough points, you *will* see the π estimate converge to the wrong value.

This is not a theoretical worry. Try it.

The fix: `erand48()` with per-thread state

`erand48()` is the reentrant cousin of `drand48()`. It takes a pointer to a caller-supplied 3-element `unsigned short` array that holds the state. Each thread keeps its own seed array on its own stack — no shared mutable state at all.

The seeding is intentionally minimal: we derive per-thread seeds from a base SEED and the thread id with simple bit operations (addition, XOR, AND). For a real simulation you'd want a proper parallel PRNG (Philox, Threefry, L'Écuyer's MRG32k3a), but for estimating π this is fine.

Ricorda: the lesson is general. Any library function that stores mutable state internally is a trap in a multi-threaded context. `strtok()`, `ctime()`, `gethostbyname()` — POSIX marks them all as "not required to be thread-safe." Always check for reentrant variants (the `_r` suffix is the POSIX convention; C11 uses `_s` suffixes).

A note on the seeding pattern across the suite

All the "correct" codes (v0, v0b, v1, v2) seed like this:

```
long int SEED = time(NULL);
int unsigned short myseeds[3] = {(short)(SEED+(myid)),
                                 (short)(SEED^(myid*3+1)),
                                 (short)(SEED&(myid*4+2))};
seed48( myseeds );
```

`use_reduction.c` instead uses a fixed `#define SEED 918273`.

`time(NULL)` as a base seed is fine for pedagogy. For **reproducibility** (and you usually want reproducibility), use a fixed seed. The mixing with `myid` is minimal — it ensures different threads get different initial states, but doesn't guarantee statistical independence of the streams. For serious Monte Carlo work (lattice QCD, molecular dynamics, ...) use a proper parallel PRNG.

`seed48` / `erand48` are POSIX, not C standard. They work on Linux and macOS; on Windows you'd need WSL or a different generator.

---

3. The four implementations

From here on we're in the performance part. All four versions are correct — they all converge to π. What differs is *how* they update the shared counter. Before running anything, let's go through them and predict what we expect.

3.1. Version 0: `critical` on every iteration

```
#pragma omp parallel
{
    // ... seed setup ...

    for( long long unsigned int i = 0; i < N; i++)
    {
        double x = erand48(myseeds);
        double y = erand48(myseeds);

       #pragma omp critical
        Nvalid += ( (x*x + y*y) < 1.0 );
    }
}
```

Every iteration, the thread enters a critical section to update the shared accumulator.
Regardless of whether the point fell inside or outside the quarter-circle.

What we expect:

A `critical` in OpenMP is a mutex — typically a futex on Linux (user-space lock with kernel fallback). Thread enters → acquires lock → all other threads block (spin or sleep) → thread exits → lock released → memory flush of shared variables.

The costs stack up:

- **Lock acquisition.** Even uncontended, a mutex acquire-release takes 20–50 ns. Comparable to a DRAM access. And it happens *every iteration*.
- **Serialisation.** Only one thread can be inside the critical section at a time. The useful work per iteration is maybe 5–10 ns (two `erand48` calls + comparison). So T−1 threads are idle most of the time. The code is effectively serial, plus the extra overhead of lock management on top.
- **Cache-line bouncing.** Nvalid is 8 bytes sitting in a 64-byte cache line. Each write pushes it to Modified state in the writer's L1 and invalidates it everywhere else. The next thread must fetch the updated line — cache-to-cache transfer on-die (30–70 cycles) or cross-socket (100+ cycles). MESI in action.
- **Memory flush.** OpenMP mandates a flush at entry and exit of a critical section. On x86 (TSO memory model) this is cheaper than on ARM, but the coherence traffic is still there. We'll ses some details in the last lecture.

Rough quantitative estimate. Say the useful work is ~10 ns/iter and the critical-section overhead is ~100 ns/iter. With T threads each doing N iterations, but only one inside the critical section at a time:

$$t_{\text{wall}} \approx N \cdot T \cdot 100\,\text{ns}$$

while without the critical section:

$$t_{\text{serial}} \approx N \cdot 10\,\text{ns}$$

The parallel version is slower than serial by a factor of roughly 10T. Adding threads worsen the scaling.

Prediction: anti-scaling. Per-thread time grows roughly linearly with T.

3.2. Version 0b: `critical` only on hits

```
for( long long unsigned int i = 0; i < N; i++)
{
    double x = erand48(myseeds);
    double y = erand48(myseeds);

    if ( (x*x + y*y) < 1.0 )
    {
       #pragma omp critical
        Nvalid++;
    }
}
```

The critical section is entered only when the point is inside the quarter-circle — about 78.5% of the time (π/4 ≈ 0.785).

Also note the difference in what's inside the critical section: v0 does `Nvalid += (x*x + y*y < 1.0)` (always enters, adds 0 or 1); v0b does `Nvalid++` (enters only when the condition is true).

Prediction: slightly better than v0. You save ~1/0.785 ≈ 1.27× in lock acquisitions. Since the lock dominates the runtime, maybe 20–25% faster. Still anti-scales — the fundamental problem is the same.

3.3. Version 1: `atomic` on every iteration

```
for( long long unsigned int i = 0; i < N; i++)
{
    double x = erand48(myseeds);
    double y = erand48(myseeds);

   #pragma omp atomic update
    Nvalid += ( (x*x + y*y) < 1.0 );
}
```

`atomic` vs. `critical` — they are not the same thing. Students often conflate the two; they shouldn't.

- A `critical` protects a *code region*. Software mutex: lock → execute whatever code → unlock → flush. Threads queue up.
- An `atomic` protects a *memory location*. On x86, for an integer addition, the compiler emits a `LOCK ADD` or `LOCK XADD` — a single hardware instruction with the LOCK prefix. The prefix holds the cache-line lock for the duration of the read-modify-write. No software mutex, no thread queue, no kernel.

There's still a cost — the LOCK prefix forces the line to Exclusive/Modified state and stalls other cores trying to access it — but it's much cheaper. A contended LOCK ADD: ~30–80 cycles. A full mutex round-trip: 100+ cycles with pipeline flushes on top.

Warning: `atomic` and `critical` do NOT protect each other. `atomic` protects memory, `critical` protects code. They're independent mechanisms. A thread doing an `atomic` on Nvalid will *not* block another thread entering a `critical` that touches the same Nvalid. **Warning:** Mix them on the same variable → possible data race.

Prediction: roughly an order of magnitude faster than v0. Still serialised at the level of the cache line — every iteration pays a contended atomic — so scalability is still poor, but better than v0.

3.4. Version 2: local accumulators — how it should be done

```
#pragma omp parallel
{
    // ... seed setup ...

    long long unsigned int local_valid_points = 0;
    for( long long unsigned int i = 0; i < N; i++)
    {
        double x = erand48(myseeds);
        double y = erand48(myseeds);

        local_valid_points += ( (x*x + y*y) < 1.0 );
    }

   #pragma omp atomic update
    Nvalid += local_valid_points;
}
```

Simple change, enormous effect. Each thread accumulates in a *private* local variable. Only at the very end — once, after all N iterations — does it touch the shared Nvalid.

Why this works. `local_valid_points` lives on the thread's stack. Private. No other thread ever reads or writes it. From the coherence point of view, the cache line holding it is Exclusive for the entire loop. No coherence traffic, no bus locks, no stalls. The increment is a plain `ADD` — about as cheap as a CPU instruction gets.

The single atomic update at the end: one LOCK ADD per thread, once. With $N = 10^6$ and T = 8, you go from 8 million contended atomics (v1) to 8. The overhead drops by a factor of N.

Prediction: near-perfect weak scaling. Per-thread time essentially constant regardless of T.

3.5. Bonus: the `reduction` clause

`pi_openmp.use_reduction.c` uses `omp for` with a `reduction` clause:

```
#pragma omp parallel
{
    // ... seed setup ...

   #pragma omp for reduction(+:Nvalid)
    for( long long unsigned int i = 0; i < N; i++)
    {
        double x = erand48( myseeds );
        double y = erand48( myseeds );

        Nvalid += ( (x*x + y*y) < 1.0 );
    }
}
```

The runtime creates a private copy of Nvalid for each thread (initialised to 0 — the identity element for addition), then combines all copies at the end of the worksharing loop. This is exactly what we did by hand in v2: local accumulator per thread, combine once.

One structural note: in v2 every thread runs the full loop `for i = 0 to N` (total points: N × nthreads). With `omp for` the iterations get *distributed*: each thread gets roughly N/T iterations (total points: N). Different semantics — make sure the normalisation matches.

Prediction: same as v2. The `reduction` clause is essentially syntactic sugar for the local-accumulator pattern. GCC and LLVM generate the same machine code.

Summary of predictions

Version	Sync per iteration	Cost per sync	Total sync cost	Scalability
v0	1 `critical` per iter	~200+ cycles (mutex + flush)	N × T × 200+ cycles	Anti-scales (worse with T)
v0b	~0.785 `critical` per iter	same	0.785 × N × T × 200+	Anti-scales

Version	Sync per iteration	Cost per sync	Total sync cost	Scalability
v1	1 `atomic` per iter	~30–80 cycles (LOCK ADD)	N × T × 30–80 cycles	Poor (contention grows with T)
v2	1 `atomic` total per thread	~30–80 cycles	T × 30–80 cycles	Near-perfect
reduction	(same as v2)	(same as v2)	T × 30–80 cycles	Near-perfect

The take is not "atomic beats critical" — though it usually does for simple operations.
The actual lesson is wider: **don't synchronise inside a tight loop if you can avoid it**, move the synchronisation out of the loops by accumulate locally and combining once at the end.

This is the most common performance pattern in shared-memory programming, and it applies far beyond this elementary π-greek example.

---

4. Try it yourselves

At this point open your laptops. Compile all four versions, run them with increasing N (10^5, 10^6, 10^7, 10^8) and increasing thread counts (1, 2, 4, 8, whatever your machine has).
Record wallclock times and per-thread times. Plot the per-thread time vs. number of threads.

You should see, roughly:

- v0 and v0b anti-scaling (time grows with T).
- v1 doing noticeably better but still getting worse with T.
- v2 staying flat.

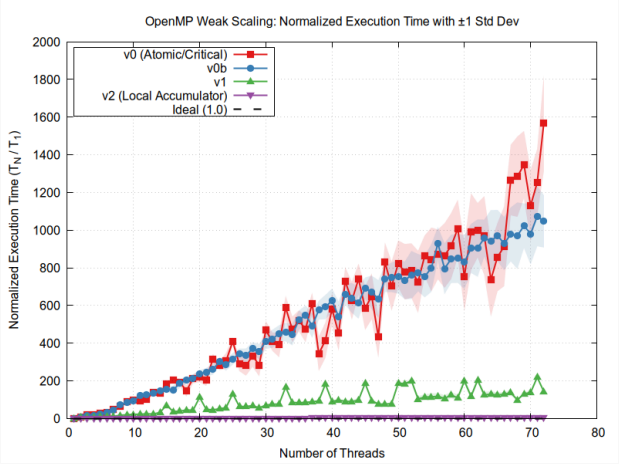
You'll also see the relative differences between versions depend on the laptop. A laptop with two physical cores + hyperthreading will look very different from an 8-core CPU. I'll now show you what it looks like on a laptop of mine, then on two very different servers — and the server comparison will teach us something we haven't covered yet.

---

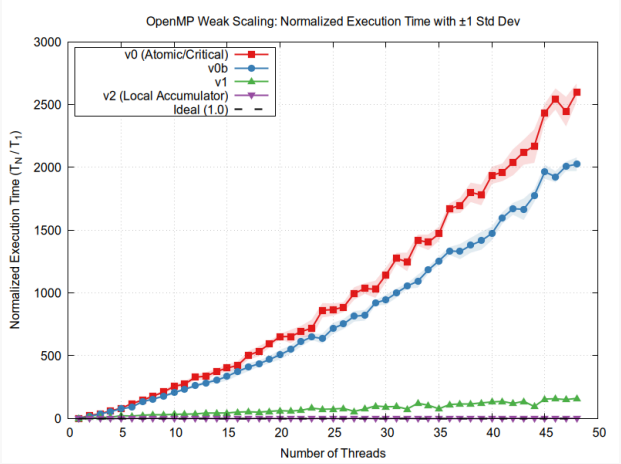
5. Results from two servers

We ran the same four versions on two machines with the same base clock (2.30 GHz) and comparable core counts, but very different topologies. Same code, same compiler, same clock.

The key plot to look at is `scalability_normalized_all.png` from each platform — per-thread execution time normalised to T_1, all four versions on one axis.



Results on the 2-sockets systems



Results on the 4-sockets system.

5.1. What the numbers say

Let's read the normalised plots at a common thread count. Take T = 36 on both machines:

Version	4-socket Gold 5118 @ 36 thr	2-socket E5-2697 v4 @ 36 thr
v0	~1500×	~600×
v0b	~1300×	~600×
v1	~100×	~60×
v2	~1×	~1×

And at the end of the range:

- **4-socket at 48 threads:** v0 ≈ 2600×, v0b ≈ 2000×, v1 ≈ 160×, v2 ≈ 1×.
- **2-socket at 36 threads:** v0 ≈ 600×, v0b ≈ 600× (indistinguishable from v0 in the noise), v1 ≈ 100–150×, v2 ≈ 1×.

Two things pop out:

1. The ordering is the same on both: v0 > v0b > v1 >> v2.
2. The contention-heavy versions are **2–3× worse** on the 4-socket system. The synchronisation-free version is **identical** on both.

Why we get such a difference with the same code and the same clock?

5.2. The two platforms

Let’s have a closer look to the two architectures as first.

Platform A: 2-socket Broadwell-EP (E5-2697 v4)

```
┌──────────────────────────────┐        QPI        ┌──────────────────────────────┐
│         Socket 0             │◄──────────────────►│         Socket 1             │
│                              │   (9.6 GT/s)       │                              │
│   18 cores, 36 threads (HT)  │    1 hop           │   18 cores, 36 threads (HT)  │
│   45 MB L3 (inclusive)       │                    │   45 MB L3 (inclusive)       │
│   4 × DDR4-2400 channels     │                    │   4 × DDR4-2400 channels     │
│                              │                    │                              │
│   Internal: dual ring bus    │                    │   Internal: dual ring bus    │
│   (9 cores per ring, ~5 cyc  │                    │   (9 cores per ring, ~5 cyc  │
│    penalty for ring-to-ring) │                    │    penalty for ring-to-ring) │
└──────────────────────────────┘                    └──────────────────────────────┘

         Total: 36 cores / 72 threads, 2 sockets
         Inter-socket: 1 direct QPI link (every socket pair: 1 hop)
```

Platform B: 4-socket Skylake-SP (Gold 5118)

```
  ┌────────────────────┐     UPI      ┌────────────────────┐
  │    Socket 0        │◄────────────►│    Socket 1        │
  │  12 cores, no HT   │              │  12 cores, no HT   │
  │  16.5 MB L3        │              │  16.5 MB L3        │
  │  (non-inclusive)   │              │  (non-inclusive)   │
  │  6 × DDR4-2400 ch  │              │  6 × DDR4-2400 ch  │
  │  Internal: mesh    │              │  Internal: mesh    │
  └────────┬───────────┘              └────────┬───────────┘
           │ UPI                               │ UPI
           │                                   │
  ┌────────┴───────────┐              ┌────────┴───────────┐
  │    Socket 3        │◄────────────►│    Socket 2        │
  │  12 cores, no HT   │     UPI      │  12 cores, no HT   │
  │  16.5 MB L3        │              │  16.5 MB L3        │
  │  (non-inclusive)   │              │  (non-inclusive)   │
  │  6 × DDR4-2400 ch  │              │  6 × DDR4-2400 ch  │
  │  Internal: mesh    │              │  Internal: mesh    │
  └────────────────────┘              └────────────────────┘

         Total: 48 cores / 48 threads (HT off), 4 sockets
         Inter-socket: ring of 2 UPI links per socket
         Socket 0 ↔ Socket 2: 2 hops (via Socket 1 or Socket 3)
```

Both are 10 years old by now. They are not the latest hardware, but they illustrate the point very cleanly. **Exercise: run the same test on a Leonardo@CINECA node.**

5.3. Why topology matters

The key difference: on Platform A, any core can reach any other core's cache in at most **1 inter-socket hop** (the direct QPI link). On Platform B, the Gold 5118 has only 2 UPI links, so in a 4-socket configuration the sockets form a *ring* — not fully connected. To get from socket 0 to socket 2 you must pass through socket 1 or 3: **2 hops**. That roughly doubles the worst-case coherence latency.

Also, 4 sockets = more participants in the coherence protocol: every write to a shared variable must potentially invalidate copies in 3 other sockets instead of 1. And the intermediate sockets' UPI links carry transit traffic on top of their own, creating congestion under contention.

Now remember what happens inside v0, v0b, and v1: every `critical` or `atomic` on Nvalid forces the cache line containing it to travel between cores. On the 4-socket system, the average path is longer and the congestion higher. On the 2-socket system, it's always one direct hop. Do this a billion times and the difference is dramatic.

In v2, Nvalid is touched exactly once per thread at the end. No billion events. No coherence traffic during the hot loop. So the topology is irrelevant — and indeed v2 gives the same result on both systems.

This is the general lesson: the cost of synchronisation is not a fixed number you can look up. It depends on the machine. Same code, same compiler, same clock — different topology, different performance. The difference lives entirely in the synchronisation.

And the corollary: **the safest way to be portable is to avoid synchronisation in the first place.** Then the topology doesn't matter.

> **NOTE — more detail on the topology:**
>
> The Gold 5118 has max 2 UPI links at 10.4 GT/s. In a 4S config, 2 UPI/socket → ring topology (0→1→2→3→0). The Platinum parts (e.g., have 3 UPI links → fully-connected "cross-bridge" topology where every pair has a direct link. This is product segmentation by Intel, not a fundamental architectural limit.
>
> The E5-2697 v4 has 2 QPI links at 9.6 GT/s; in 2S, both go to the other socket — one 1-hop path.
>
> On-die interconnect also differs. Broadwell-EP: dual ring bus with a buffered switch, ~5-cycle penalty for cross-ring. Skylake-SP: mesh interconnect, runs at 1.8–2.4 GHz (vs. the ring's ~3 GHz) but offers shortest-path routing. For a 12-core die the mesh is small, so on-die latency is not dramatically different — ~10% higher average L3 latency according to Intel's own numbers.
>
> The Skylake-SP L3 is non-inclusive (more like a victim cache); the Broadwell-EP L3 is inclusive and doubles as a snoop filter. In Skylake-SP, each mesh stop has a CHA (Caching and Home Agent) with a distributed snoop filter. For this workload, the dominant factor is the inter-socket hop count, not the L3 topology.

> **SIDE NOTE — secondary factors in the platform comparison:**
>
> 1. *Coherence participants.* 4 sockets → directory-based protocol must track more sharers; each CHA may need to invalidate up to 3 remote sockets vs. 1.
> 2. *UPI link congestion.* In the 4S ring, each link carries local + transit traffic. Under contention, they compete for bandwidth. 2S: no transit.
> 3. *On-die interconnect.* Broadwell dual ring: ~30 cy same-ring,

> ~35 cy cross-ring. Skylake mesh with 12 cores: comparable worst-case hop count but lower mesh clock. Secondary.
>
> 4. *L3 topology.* Broadwell inclusive L3 = perfect snoop filter. Skylake non-inclusive L3 with distributed CHA snoop filters. Single hot line → probably not dominant.
> 5. *Turbo.* E5-2697v4 turbos to 3.6 GHz, Gold 5118 to 3.2 GHz. Affects single-thread baseline slightly.

5.4. Details to notice in the individual plots

Once the big picture is clear, a few details in the per-version plots are worth a look.

v0 scalability plots, both platforms. The standard-deviation bands are very tight — all threads are equally slow. The mutex serialises them so completely that everyone waits about the same amount.

v1 on 2-socket (`scalability.v1.png`, Broadwell). Striking periodic spikes — sharp upward jumps at roughly regular intervals. These are a fingerprint of the dual-ring topology: when thread placement forces traffic across the ring-to-ring bridge, or across the socket boundary, latency jumps.
The 4-socket v1 plot is smoother — the mesh distributes traffic more evenly, even though its average latency is higher.

> **SIDE NOTE — possible interpretation of the v1 spikes on Broadwell:**
>
> Broadwell-EP has two 9-core rings connected by a buffered switch (~5-cycle penalty). When the OS scheduler puts threads such that the contended line crosses the ring bridge more often, per-atomic cost jumps. When threads spill onto socket 1, the cost jumps again (QPI hop: ~100+ cycles vs. intra-socket: ~30–50 cycles). The spikes at ~T = 14, 19, 26, 33, 38 probably correspond to placement patterns that maximise cross-ring or cross-socket coherence traffic.
>
> The mesh-based 4-socket system grows more smoothly — shortest-path routing makes the cost more uniform across placements — but its *average* cost is higher because of the 2-hop inter-socket topology.

v2 efficiency plot, 4-socket. Shows T_1/T_N *above* 1.0 — around 1.1–1.3. "Super-linear weak scaling" sounds exciting, but it's just **turbo boost**. With 1 thread, the sole active core runs at 3.2 GHz (Gold 5118 turbo). With 48 threads, everybody runs at the base 2.3 GHz. So T_1 was measured at a faster clock than T_N, and the ratio comes out above 1.

v2 efficiency plot, 2-socket. Tells a different story: ~0.83–0.85 up to 36 threads, then drops linearly to ~0.42 at 72 threads. The knee at 36 is the hyperthreading boundary — beyond that, two hyperthreads share each core's execution resources. The HT factor works out to ~0.42/0.83 ≈ 0.5, i.e. each hyperthread gets about half the throughput of a dedicated core.

> **SIDE NOTE — super-linear scaling:**
>
> **"super-linear"** in **weak** scaling is almost always a measurement artifact (frequency, NUMA placement).
> **"Super-linear"** in **strong** scaling can be real: if the problem size per core shrinks enough, the working set drops into a faster
> cache level and throughput actually increases.

6. Hardware mechanisms

Some connection to the architecture lectures.

6.1. Cache-line bouncing (MESI)

In v0 and v1, Nvalid sits in one cache line. Every write pushes it to M(odified) in the writer's L1 and invalidates it everywhere else. The next thread must fetch the updated line — via cache-to-cache transfer on-die (30–50 cycles) or cross-socket (100–200 cycles, double that for a 2-hop path).

In v2, the line with Nvalid is touched once per thread at the end. Each thread's local counter stays Exclusive the whole time. No bouncing.

Note: False sharing

In v2, `local_valid_points` is on the thread's stack — stacks are separated by at least a page, so no false sharing. But imagine the wrong way to do it:

```
long long unsigned int counts[nthreads];
// ...
#pragma omp parallel
{
    int tid = omp_get_thread_num();
    for (...) {
        counts[tid] += (...);
    }
}
```

With a small `nthreads`, the whole array fits in one or two cache lines. Threads write to different elements, but those elements share a line. You get bouncing even though no two threads touch the same variable. This is **false sharing**, and it can be as bad as true sharing.
Fix: pad each element to a full cache line (64 bytes), or better, just use stack-local variables.

---

7. Exercises and discussion

Exercises

1. **Run the benchmark suite** on your laptop with increasing N and T. Record wallclock times and per-thread times.
2. **Plot weak scaling** for each version: per-thread time vs. thread count. v0 and v0b should anti-scale; v1 should grow more slowly; v2 should stay flat.
3. **Implement a false-sharing version**: replace the local accumulator in v2 with a shared array `counts[nthreads]`, indexed by thread id. Measure the degradation. Then add padding (`__attribute__((aligned(64)))` or a padding struct) and measure again.
4. **Use `perf stat`** or `likwid-perfctr` to measure cache misses and bus transactions for v0 vs. v2. You should see dramatically higher L1/LLC miss rates and interconnect traffic in v0.
5. **RUN THE BENCHMARK ON A LEONARDO NODE** (different socket count, different interconnect topology).

8. Summary

This exercise is small, but conveys a general principle:
minimise synchronisation.

OpenMP gives you `critical`, `atomic`, `reduction`, local accumulators — from maximally expensive to essentially free. The cost hierarchy comes from cache coherence, bus locking, and mutex overhead.

"Accumulate locally, reduce once" is not a trick. It's the default design for any parallel reduction. If you see an `atomic` or a `critical` inside a tight loop, treat it as a red flag: there's almost certainly a way to push the synchronisation out of the loop.

And from the platform comparison: **synchronisation costs are not portable.**
Same code, same clock, different topology → very different performance for contention-heavy code.
The only robust strategy is to eliminate unnecessary synchronisation altogether. Then the topology doesn't matter.

Cross-references: MESI protocol and cache coherence → Intro to Parallelism slides 81–85.
`critical`, `atomic`, `reduction` semantics → Parallel Regions slides 35–45.
False sharing and variable classification → Intro to Parallelism slide 85.

Reference plots

`OpenMP/examples/loops/pi_greek/weak_scalability/2sockets_Xeon_BroadwellEP`
`OpenMP/examples/loops/pi_greek/weak_scalability/4sockets_Xeon_SkyLakeSP_Gold/`

4-socket Intel Xeon Gold 5118 (Skylake-SP, 48 cores, HT off):
`scalability_normalized_all.png` — all four versions compared (the key plot),
`scalability.v0.png` through `scalability.v2.png` — absolute per-thread time,
`efficiency.v0.png` through `efficiency.v2.png` — T_1/T_N ratio,
`speedup.v0.png` through `speedup.v2.png` — T_N/T_1 ratio.

2-socket Intel Xeon E5-2697 v4 (Broadwell-EP, 36 cores, HT on → 72 threads):
Same set. Thread counts beyond 36 use hyperthreading; the v2 efficiency plot shows the HT knee at T = 36.